

A COOKBOOK FOR ENGINEERS

Reinforcement Learning Cookbook

Teach software to act — how an agent learns which decisions pay off, from the core loop to shipping a policy, with Gymnasium and Stable-Baselines3.

Gymnasium

Stable-Baselines3

MDPs

Q-learning

DQN

PPO

Reward design

PyTorch

Written for a developer who knows Python and some ML, and now wants the 'act on information' layer — agents that decide, not just predict.

Every chapter answers: what it is, how to build it in Python, the pitfalls, and when NOT to use RL at all.

Edition 2026 · intuition over proofs · policies that ship

Table of Contents

How to Use This Book	4
The one idea: the agent-environment loop	4
What this book covers	5
Part 1 • Is This Even a Reinforcement Learning Problem?	6
1.1 The three families, mapped	6
1.2 The five conditions RL needs	6
1.3 When NOT to use RL	7
1.4 Where RL genuinely shines	7
Part 2 • The Formalism: Markov Decision Processes	8
2.1 The five pieces	8
2.2 Return, discounting, and the goal	8
2.3 Policies and value functions	9
2.4 The exploration-exploitation trade-off	9
Part 3 • The Toolkit: Gymnasium & Stable-Baselines3	10
3.1 The Gymnasium environment API	10
3.2 Running a real algorithm with SB3	10
3.3 Writing your own environment	11
3.4 Vectorised environments	11
Part 4 • Multi-Armed Bandits — RL Without the Hard Part	13
4.1 The setup	13
4.2 Three strategies	13
4.3 Contextual bandits — halfway to RL	14
Part 5 • Value-Based Learning: Q-Learning	15
5.1 The Q-table	15
5.2 The update rule	15
5.3 Acting while learning: ϵ -greedy	16
5.4 On-policy vs off-policy (SARSA vs Q-learning)	16
Part 6 • Deep Q-Networks (DQN)	17
6.1 The idea	17
6.2 The two tricks that make it work	17
6.3 In practice — use SB3	17
6.4 What DQN can and can't do	18
Part 7 • Policy-Based Methods & PPO	19
7.1 Why learn the policy directly	19
7.2 The policy-gradient idea	19
7.3 PPO — the workhorse	19
7.4 The other names you'll meet	20
Part 8 • Reward Design — Where RL Projects Live or Die	21
8.1 The agent optimises the reward, not your intent	21
8.2 Sparse vs shaped rewards	21

8.3 Practical rules	21
Part 9 · Evaluation, Debugging & Stability	23
9.1 Evaluate separately from training	23
9.2 Seeds and variance	23
9.3 A debugging ladder	23
9.4 Overfitting to the simulator	24
Part 10 · RL in Production	25
10.1 Serving a policy is easy — it's just inference.	25
10.2 The safety problem: offline vs online.	25
10.3 Drift — the world stops matching the simulator	26
10.4 Realistic production patterns.	26
Part 11 · Capstone & Cheat Sheet	28
11.1 Capstone — take one agent end to end	28
11.2 Decision cheat sheet.	28
11.3 Concept cheat sheet	28
11.4 The five rules.	29

How to Use This Book

Your Machine Learning cookbook taught prediction: show a model labelled examples, it learns to predict the label. But prediction alone doesn't *do* anything — it tells you the churn probability; it doesn't decide who to call. **Reinforcement learning (RL)** is the missing layer: how a program learns which *actions* to take, over time, to maximise a goal — with no labelled "right answer," only a reward signal telling it how well it's doing.

This is the "act on information" part of intelligent systems. Where supervised learning learns a function from inputs to outputs, RL learns a **policy**: a strategy mapping situations to actions, improved by trial, error, and reward.

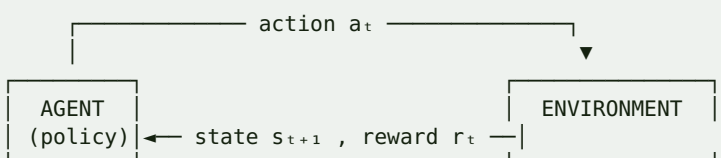
Each chapter answers four questions:

1. **What is this and when do you use it?**
2. **How do you build it in Python?**
3. **What are the pitfalls?**
4. **When is RL the wrong tool?** (This matters more in RL than anywhere else.)

The one idea: the agent-environment loop

Every RL problem is the same loop. An **agent** observes the **state** of an **environment**, picks an **action**, and receives a **reward** plus the next state. It repeats, learning a **policy** that earns the most reward over time.

DIAGRAM



That's it. Perceive the state, act, get rewarded, update the strategy, repeat. Chess, a warehouse robot, a trading bot, an ad-bidding system, a game AI, a recommendation sequencer — all the same shape.

ENGINEER'S MENTAL MODEL

A policy is a function `action = policy(state)` — the thing you're trying to learn. Unlike supervised learning, nobody hands you `(state, best_action)` pairs. Instead you get a scalar reward *after* acting, often much later, and must figure out which earlier actions deserve the credit. That "credit-assignment over time" is the whole difficulty, and the whole point.

What this book covers

The mental model (agents, environments, rewards, MDPs), the toolkit (Gymnasium + Stable-Baselines3), the classic algorithm families — **value-based** (Q-learning, DQN) and **policy-based** (policy gradients, PPO) — and the parts that decide whether RL works in practice: **reward design**, **exploration**, **evaluation**, and **when not to use RL at all**. It favours intuition and working code over proofs.

HOW TO READ IT

Read Parts 1-3 in order — the loop, the MDP formalism, and the toolkit are the foundation. Then Parts 4-7 build up the algorithms from simplest (bandits) to the workhorse (PPO). Parts 8-10 are the practitioner's reality: reward design, evaluation, and production. Do the exercises with Gymnasium actually installed — RL is learned by watching an agent's reward curve climb.

READ THIS FIRST: RL IS OFTEN THE WRONG TOOL

RL is powerful and *expensive* — it needs a simulator or cheap interaction, millions of steps, and careful reward design, and it's notoriously unstable. If a problem can be solved by a **rule**, a **forecast** (supervised learning), or an **optimiser** (operations research), do that instead. Part 1 makes the "is this even an RL problem?" test explicit. Most business problems are not RL problems — and knowing that will save you months.

Part 1 · Is This Even a Reinforcement Learning Problem?

The most valuable RL skill is recognising when *not* to use it. RL is the right tool for a narrow, specific shape of problem; forced onto anything else it's slow, unstable, and beaten by simpler methods. This chapter gives you the test.

1.1 The three families, mapped

You need to...	Field	Example
Predict a value/label from features	Supervised learning	churn probability, price forecast
Find structure with no labels	Unsupervised learning	customer segments, anomalies
Choose a sequence of actions to maximise a long-term goal	Reinforcement learning	game play, robot control, bidding

RL is uniquely about **sequential decisions under a delayed reward**. If your problem isn't sequential, or the "reward" is immediate and labelled, you almost certainly want supervised learning.

1.2 The five conditions RL needs

Reach for RL only when *most* of these hold:

1. **Sequential decisions** — actions now affect the situations (and rewards) you face later. One-shot decisions are a prediction problem, not RL.
2. **A goal expressible as a reward** — you can put a number on "good" (score, profit, time-to-goal).
3. **Cheap interaction** — you have a **simulator**, a game, or a system you can safely poke millions of times. RL learns by trying; trying is the cost.
4. **No labelled optimal actions** — if you *knew* the best action for each state, you'd train a classifier (imitation learning) instead.
5. **Tolerance for exploration** — the agent will do dumb things while learning. Fine in a simulator; often unacceptable on live users or real hardware.

ENGINEER'S MENTAL MODEL

Condition 3 is the one that kills most business RL projects. RL is sample-hungry — modern algorithms routinely need millions of interactions. That's fine in a simulator (a game, a physics engine, a model of your system), and a non-starter if each "step" is a real customer, a real trade, or a real robot arm that can break. No cheap environment → no RL.

1.3 When NOT to use RL

WHEN NOT TO USE RL

Don't use RL when: a **rule** works ("reorder when stock < N"); a **forecast** plus a simple decision rule works (predict demand, then order to it); the decision is **one-shot** (that's classification/regression); you can only interact with the **real world** and mistakes are costly and you have no simulator; or you have **logged optimal behaviour** to imitate (use supervised imitation learning). RL earns its keep on genuinely sequential control problems with a cheap environment — games, simulations, robotics, long-horizon optimisation — not on most tabular business decisions.

COMMON MISTAKE

Framing a forecasting problem as RL because "it makes decisions." Deciding how much inventory to hold is usually *predict demand, then optimise the order* — a supervised model feeding a simple rule or an operations-research optimiser. Wrapping it in an agent-environment loop adds instability and sample cost for no benefit. Ask first: "could a forecast plus a rule do this?" If yes, it's not an RL problem.

1.4 Where RL genuinely shines

Games (Atari, Go, StarCraft), robotics and continuous control, resource scheduling and routing with long horizons, real-time bidding, recommendation as a *sequence* (not one click), and — increasingly — using RL to fine-tune other models (RLHF for LLMs is reinforcement learning with human preference as the reward).

EXERCISE 1.1

Take three decision problems from your own work. For each, run the five-point test: is it sequential? is there a reward? can you interact cheaply? do you lack labelled optimal actions? can you tolerate exploration? Classify each as supervised, unsupervised, RL, or "just a rule/optimiser." Be honest — if none come out as RL, that's the correct and common answer.

Part 2 · The Formalism: Markov Decision Processes

RL has one core abstraction, the **Markov Decision Process (MDP)**. You don't need the measure theory, but you do need the vocabulary — every algorithm is defined in these terms, and a fuzzy grasp here makes everything later fuzzy too.

2.1 The five pieces

An MDP is (S, A, P, R, γ) :

Symbol	Name	Meaning
S	states	the situations the agent can be in
A	actions	what it can do
P	transition	$P(s' \mid s, a)$ — where you land after acting
R	reward	$R(s, a)$ — the scalar feedback
γ	discount	how much future reward counts now (0-1)

The **Markov property**: the next state depends only on the *current* state and action, not the full history. If your problem needs memory (the past matters), you fold that history into the state — that's your job when you design the state representation.

2.2 Return, discounting, and the goal

The agent doesn't maximise the next reward — it maximises the **return**, the (discounted) sum of *all* future rewards:

DIAGRAM

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \dots$$

γ (gamma, typically 0.95–0.99) makes near rewards worth more than distant ones and keeps the sum finite. $\gamma=0$ is myopic (only the next reward); $\gamma \rightarrow 1$ is far-sighted.

ENGINEER'S MENTAL MODEL

γ is a discount rate, exactly like net present value in finance: a reward ten steps away is worth γ^{10} of the same reward now. Tuning it changes the agent's personality — low γ makes a greedy short-termist, high γ makes a patient planner that sacrifices now for later. It's one of your most important knobs.

2.3 Policies and value functions

Two objects you'll meet in every algorithm:

- **Policy** $\pi(a|s)$ — the agent's strategy: what to do in each state. This is what you ultimately want.
- **Value functions** — how good things are:
- $V(s)$ — expected return starting from state s .
- $Q(s, a)$ — expected return from taking action a in state s , then following the policy. This one is central: if you know Q , the best action is just $\operatorname{argmax}_a Q(s, a)$.

DIAGRAM

best action in state $s = \operatorname{argmax} \text{ over } a \text{ of } Q(s, a)$

ENGINEER'S MENTAL MODEL

$Q(s, a)$ is a lookup table (or a learned function) answering "if I do a here, how much total reward do I expect?" Learn a good Q and the policy falls out for free — you just pick the highest- Q action. That's the entire idea behind value-based methods (Parts 5–6).

2.4 The exploration-exploitation trade-off

The defining dilemma of RL: do you **exploit** (take the action you currently believe is best) or **explore** (try something else to learn whether it's better)? Too much exploitation and you lock onto a mediocre habit; too much exploration and you never cash in. Every algorithm needs an answer; the simplest is **ϵ -greedy** (act greedily, but with probability ϵ pick a random action).

COMMON MISTAKE

Designing a state that violates the Markov property and then wondering why learning stalls. If the optimal action depends on something not in the state (the last three prices, whether a door was already opened), the agent literally cannot learn it. Fix it by **putting that information into the state** — stack recent frames, add the needed history — rather than blaming the algorithm.

EXERCISE 2.1

Write out the MDP for a tic-tac-toe agent: what are S , A , R , and a sensible γ ? Then do the same for "a thermostat keeping a room at 21°C." For each, state one way you might accidentally break the Markov property and how you'd fix it by enriching the state.

Part 3 · The Toolkit: Gymnasium & Stable-Baselines3

You won't implement most algorithms from scratch in practice — you'll use two libraries. **Gymnasium** (the maintained successor to OpenAI Gym) is the standard way to *define and interact with* environments. **Stable-Baselines3 (SB3)** gives you reliable, tested implementations of the major algorithms. Learn the interface and you can run real RL today.

3.1 The Gymnasium environment API

Every environment exposes the same tiny interface — this is the agent loop from the intro, in code:

PYTHON

```
import gymnasium as gym

env = gym.make("CartPole-v1")
obs, info = env.reset(seed=0)           # start an episode -> first observation

for _ in range(1000):
    action = env.action_space.sample()   # a random policy (placeholder)
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        # episode ended (goal, failure, or time limit)
        obs, info = env.reset()
env.close()
```

- `reset()` → starts an episode, returns the first observation.
- `step(action)` → returns `(obs, reward, terminated, truncated, info)`.
- `action_space` / `observation_space` describe the shapes (discrete vs continuous).

ENGINEER'S MENTAL MODEL

`step()` is one turn of the loop: you hand it an action, it returns the reward and next state. `terminated` means the episode reached a natural end (won, crashed); `truncated` means it hit a time limit. This five-value tuple is the contract *every* RL algorithm consumes — learn it once and every environment looks the same.

3.2 Running a real algorithm with SB3

Training a competent agent is a few lines — SB3 handles the networks, replay buffers, and update math:

PYTHON

```

from stable_baselines3 import PPO
import gymnasium as gym

env = gym.make("CartPole-v1")
model = PPO("MlpPolicy", env, verbose=1)      # PPO with a small neural-net policy
model.learn(total_timesteps=100_000)         # train
model.save("cartpole_ppo")

# use the trained policy
obs, _ = env.reset()
for _ in range(1000):
    action, _ = model.predict(obs, deterministic=True)  # exploit the learned policy
    obs, reward, terminated, truncated, _ = env.step(action)
    if terminated or truncated:
        obs, _ = env.reset()

```

PRODUCTION TIP

Start every project by getting a **stock SB3 algorithm running on a stock Gymnasium environment** end to end before you touch anything custom. It proves your install, your loop, and your evaluation work. Only then swap in your own environment. Debugging a custom env and custom training at once is how RL projects stall.

3.3 Writing your own environment

Your real problem becomes an RL problem when you wrap it as a Gymnasium `Env`: define the observation space, the action space, `reset()`, and `step()` (which returns the reward). If you can write those four things, SB3 can train on it.

PYTHON

```

class InventoryEnv(gym.Env):
    def __init__(self):
        self.action_space = gym.spaces.Discrete(100)      # order 0..99 units
        self.observation_space = gym.spaces.Box(low=0, high=1e4, shape=(3,))
    def reset(self, seed=None, options=None): ...          # -> (obs, info)
    def step(self, action): ...                             # -> (obs, reward,
term, trunc, info)

```

3.4 Vectorised environments

RL is sample-hungry, so SB3 runs **many copies of the environment in parallel** (`make_vec_env`) to collect experience faster. It's the standard way to cut wall-clock training time — the same "throw more workers at it" instinct as your queues, applied to simulation.

COMMON MISTAKE

Reinventing the algorithms. Beginners hand-code DQN, get the target-network or replay-buffer details subtly wrong, and conclude "RL doesn't work." Use SB3's tested implementations for anything real; implement from scratch only to *learn* (Parts 5–7 show the ideas), not to ship. RL has a hundred small correctness details and SB3 already got them right.

EXERCISE 3.1

Install `gymnasium` and `stable-baselines3`, then train PPO on `CartPole-v1` for 100k steps and watch the episode reward climb toward 500. Save the model, reload it, and run it with `deterministic=True`. Then swap the environment for `LunarLander-v2` and see what changes (and what doesn't) in your code.

Part 4 · Multi-Armed Bandits — RL Without the Hard Part

Before full RL, meet its simpler cousin: the **multi-armed bandit**. It strips away the sequential part (no state transitions) and leaves the pure exploration–exploitation dilemma. It's genuinely useful on its own — A/B testing, recommendations, ad selection — and it's the gentlest on-ramp to RL thinking.

4.1 The setup

You face k slot machines ("arms"), each with an unknown payout rate. Each round you pull one arm and see its reward. Goal: maximise total reward over many rounds. There's **no state** — every round looks the same — so the only question is the one RL question in miniature: exploit the arm that looks best, or explore to learn more?

DIAGRAM

```
round 1: pull arm 3 → reward 0
round 2: pull arm 1 → reward 1
round 3: pull arm 1 → reward 1
...
```

which arm is truly best?
spend pulls finding out (explore)
or cash in on the leader (exploit)

ENGINEER'S MENTAL MODEL

A bandit is an **adaptive A/B test**. Classic A/B testing splits traffic evenly for a fixed period, then picks a winner — wasting traffic on the losing variant the whole time. A bandit shifts traffic toward the winner *as it learns*, cutting the cost of experimentation. Same math, framed as regret minimisation.

4.2 Three strategies

- **ϵ -greedy** — exploit the current best arm, but with probability ϵ pick a random arm. Dead simple, works surprisingly well.
- **UCB (Upper Confidence Bound)** — pick the arm with the best *optimistic* estimate (value + an uncertainty bonus). Explores arms it's unsure about.
- **Thompson sampling** — keep a probability distribution over each arm's value, sample from each, pick the winner. Elegant and strong in practice.

PYTHON

```
import numpy as np
def epsilon_greedy(values, counts, eps=0.1):
    if np.random.random() < eps:
        return np.random.randint(len(values))    # explore
    return int(np.argmax(values))                # exploit
```

```
# after pulling arm a and seeing reward r, update the running mean:
#   counts[a] += 1
#   values[a] += (r - values[a]) / counts[a]
```

4.3 Contextual bandits — halfway to RL

Add a **context** (features about the current situation) and you get a **contextual bandit**: pick the best arm *given* this user/context. Now it's a prediction problem per round — "which action has the highest expected reward given these features" — and it powers a lot of real recommendation and ad systems. It's still not full RL because actions don't change future state.

PRODUCTION TIP

For "which of N options should I show this user right now?", a **contextual bandit is very often the right tool — not full RL**. It's far more sample-efficient, stable, and safe to run on live traffic, and it directly optimises the click/convert you care about. Reach for full sequential RL only when today's action genuinely changes tomorrow's state.

WHEN NOT TO USE A BANDIT

If one variant is clearly better and won't change, a bandit is overkill — just ship the winner. And if you need a *statistically clean* comparison for a stakeholder decision (a p-value, a confidence interval on the lift), a classic fixed A/B test is easier to reason about than a bandit that's been quietly reallocating traffic.

COMMON MISTAKE

Setting ϵ to a fixed high value forever, so the system keeps wasting a chunk of traffic on known-losing arms indefinitely. Decay ϵ over time (explore a lot early, exploit later), or use UCB/Thompson which handle the trade-off automatically instead of at a fixed rate.

EXERCISE 4.1

Simulate a 5-arm bandit with fixed hidden payout probabilities. Implement ϵ -greedy and compare total reward for $\epsilon \in \{0, 0.01, 0.1, 0.5\}$ over 1000 rounds. Add a decaying ϵ and Thompson sampling. Plot cumulative reward and see which strategy finds and exploits the best arm fastest.

Part 5 · Value-Based Learning: Q-Learning

Now bring back state. **Q-learning** is the classic tabular RL algorithm and the one to understand deeply, because everything modern (DQN, and much else) is built on its idea: learn $Q(s, a)$ — the value of each action in each state — then act greedily with respect to it.

5.1 The Q-table

For small, discrete problems, Q is literally a table: rows are states, columns are actions, cells are "expected return if I take this action here." Start it at zero and fill it in through experience.

DIAGRAM

	action: left	right	up	down	
state A	0.0	1.2	0.3	0.0	
state B	2.1	0.0	0.0	0.5	← in state B, "left" looks best

5.2 The update rule

After taking action a in state s , seeing reward r and next state s' , nudge $Q(s, a)$ toward the reward plus the best you can do next:

DIAGRAM

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \left[\underbrace{r + \gamma \cdot \max_{a'} Q(s', a')}_{\text{target}} - \underbrace{Q(s, a)}_{\text{current}} \right]$$

α is the learning rate; the bracket is the **temporal-difference (TD) error** — the gap between what you predicted and what you now estimate. Shrinking that gap, step after step, is learning.

PYTHON

```
q = np.zeros((n_states, n_actions))
def update(s, a, r, s_next, alpha=0.1, gamma=0.99):
    target = r + gamma * q[s_next].max()
    q[s, a] += alpha * (target - q[s, a])
```

ENGINEER'S MENTAL MODEL

The TD error is a prediction error you're doing gradient descent on — same instinct as any ML loss, but the "label" is *bootstrapped* from your own current estimate of the next state's value ($\gamma \cdot \max_a Q(s', \cdot)$), not a ground-truth answer. Learning from your own estimates is what makes RL powerful and what makes it unstable (Part 6).

5.3 Acting while learning: ϵ -greedy

Q-learning needs exploration to discover good actions, so you act ϵ -greedily on the current table: mostly take `argmax_a Q(s,a)`, occasionally act randomly. Decay ϵ over training — explore early, exploit late.

5.4 On-policy vs off-policy (SARSA vs Q-learning)

A distinction you'll see everywhere: **Q-learning is off-policy** — it learns the value of the *best* action (`max`) regardless of what it actually did, so it can learn the optimal policy while exploring. **SARSA is on-policy** — it learns the value of the action it *actually took next*, so it accounts for its own exploration (more conservative near cliffs). Off-policy is more flexible; on-policy is often safer.

COMMON MISTAKE

Expecting tabular Q-learning to scale. A Q-table needs a row per distinct state — fine for a grid world, hopeless for images or continuous state (the table would be astronomically large or infinite). The moment your state is high-dimensional or continuous, you must **approximate Q** with a function (a neural net) instead of tabulating it — that's DQN, next chapter.

WHEN NOT TO LEAVE THE TABLE BEHIND

Conversely, don't reach for deep RL when the problem is small and discrete. A tabular method is faster, more stable, and fully interpretable for modest state spaces. Add a neural network only when the state is too big to tabulate — not because it sounds more impressive.

EXERCISE 5.1

Implement tabular Q-learning on `FrozenLake-v1` (a small discrete grid). Use ϵ -greedy with decaying ϵ , the update rule above, and $\gamma=0.99$. Plot success rate over episodes and watch it climb. Then switch the update to SARSA and compare behaviour on the slippery version — note which is more cautious.

Part 6 · Deep Q-Networks (DQN)

When the state is too large to tabulate — images, many continuous variables — replace the Q-table with a **neural network** that approximates $Q(s, a)$. That's a **Deep Q-Network**, the algorithm that first made RL play Atari from pixels. It's Q-learning (Part 5) with a network in place of the table, plus two tricks that keep it from falling apart.

6.1 The idea

The network takes a state and outputs a Q-value for each action:

DIAGRAM

```
state s → [ neural net  $\theta$  ] → [  $Q(s, \text{left})$ ,  $Q(s, \text{right})$ ,  $Q(s, \text{up})$ ,  $Q(s, \text{down})$  ]
act with argmax; train the net so its outputs match the TD target
```

You train it by gradient descent on the same TD error as tabular Q-learning, now as a loss over the network's weights θ .

ENGINEER'S MENTAL MODEL

DQN is "supervised learning where you generate your own labels." Each step produces a training example: input = state, target = $r + \gamma \max_a Q(s', a)$. You fit the network to those targets like any regression. The catch is the targets move as the network learns and the data isn't i.i.d. — the next two tricks exist to fix exactly that.

6.2 The two tricks that make it work

- **Experience replay** — store transitions (s, a, r, s') in a buffer and train on random minibatches from it. This breaks the correlation between consecutive samples (RL data is a highly correlated stream) and reuses experience efficiently.
- **Target network** — keep a second, slowly-updated copy of the network to *compute the targets*. If you used the live network for both prediction and target, you'd be chasing a moving goal and training would diverge. The frozen target stabilises it.

6.3 In practice — use SB3

You almost never hand-roll DQN; the details (buffer size, target-update cadence, ϵ schedule) are fiddly and SB3 has them right:

PYTHON

```
from stable_baselines3 import DQN
import gymnasium as gym
```

```
model = DQN("MlpPolicy", gym.make("CartPole-v1"),
            buffer_size=50_000, learning_starts=1_000,
            target_update_interval=500, exploration_fraction=0.2,
            verbose=1)
model.learn(total_timesteps=100_000)
```

6.4 What DQN can and can't do

DQN handles **high-dimensional states** (with a CNN policy it learns from pixels) but only **discrete action spaces** — it picks the argmax over a fixed set of actions. For **continuous** actions (a steering angle, a throttle), you need policy methods (Part 7) or continuous-action value methods (DDPG/SAC), because you can't `argmax` over an infinite action set.

COMMON MISTAKE

Reaching for DQN on a continuous-control problem. If the action is a real number (or vector) — torque, price, angle — DQN doesn't apply without crude discretisation. Use PPO or SAC instead. Matching the algorithm family to the action space (discrete → DQN/PPO; continuous → PPO/SAC) is a basic but common mismatch.

PRODUCTION TIP

Watch the **loss and the episode-reward curve together**. In DQN a falling loss with flat reward often means the network is confidently learning the wrong targets (a bug in the reward, the done-flag, or the target network). Reward-up is the only success signal that matters; loss-down alone can lie.

EXERCISE 6.1

Train SB3's DQN on `CartPole-v1` and compare its learning curve to the PPO run from Part 3. Then try it on an image-based env with a `CnnPolicy`. Finally, attempt a continuous-action env and observe that DQN doesn't fit — then swap to PPO and confirm it does. That contrast is the discrete-vs-continuous lesson in your hands.

Part 7 · Policy-Based Methods & PPO

Value methods learn Q and *derive* a policy from it. **Policy-based methods** skip that and learn the policy directly — a network that outputs actions (or a distribution over them). This is what powers most modern RL, and one algorithm — **PPO** — is the reliable default you'll reach for again and again.

7.1 Why learn the policy directly

- **Continuous actions.** A policy network can output a real-valued action (or the mean/spread of a distribution), so it handles steering, torque, or price — where `argmax` over actions is impossible.
- **Stochastic policies.** It can learn to act *randomly on purpose* (useful when the best strategy is unpredictable, e.g. bluffing), which value methods can't represent naturally.
- **Smoother learning.** It nudges action probabilities gradually instead of jumping when one Q-value overtakes another.

DIAGRAM

state $s \rightarrow$ [policy net θ] $\rightarrow$ a distribution over actions $\rightarrow$ sample an action
 push up the probability of actions that led to high return (policy gradient)

7.2 The policy-gradient idea

The core trick: run the policy, see the returns, then adjust the network to make **high-return actions more likely and low-return actions less likely**. That's the policy gradient. Pure versions (REINFORCE) are high-variance and unstable; the practical algorithms add a **critic** (a value estimate) to reduce variance — these are **actor-critic** methods (the actor picks actions, the critic judges them).

ENGINEER'S MENTAL MODEL

Actor-critic is two networks in a feedback pair: the **actor** is the policy (`what do I do?`), the **critic** estimates value (`how good was that, really?`). The critic's judgement tells the actor which way to adjust — a learned, lower-variance version of "reward good actions." A2C, PPO, and SAC are all flavours of this.

7.3 PPO — the workhorse

Proximal Policy Optimization (PPO) is the algorithm most practitioners start *and* finish with. It's an actor-critic method whose one big idea is **don't change the policy too much in a single update** (a "clip" that keeps each step close to the previous policy). That

constraint is what makes it stable and forgiving — it just works across a huge range of problems, which is why it's the default.

PYTHON

```
from stable_baselines3 import PPO
import gymnasium as gym

model = PPO("MlpPolicy", gym.make("LunarLander-v2"),
            n_steps=2048, batch_size=64, gae_lambda=0.95,
            gamma=0.99, ent_coef=0.0, verbose=1)
model.learn(total_timesteps=500_000)
```

7.4 The other names you'll meet

- **A2C** — simpler synchronous actor-critic; fast, less sample-efficient.
- **SAC / TD3** — strong for **continuous control**, very sample-efficient (off-policy); favourites in robotics.
- **PPO** — the robust general default, especially when you're unsure.

PRODUCTION TIP

Start with PPO. It's the least likely to silently fail, tolerates imperfect hyperparameters, and works on discrete and continuous actions alike. Only move to SAC/TD3 when you specifically need continuous-control sample efficiency, or to DQN-family methods for cheap discrete problems. Switching algorithms is rarely the fix for a struggling agent — the reward and the environment usually are (Part 8).

COMMON MISTAKE

Blaming the algorithm for slow learning and cycling through PPO → SAC → DQN hoping one "works." In practice the problem is almost always the **reward design, the observation, or a bug in the environment** — not the algorithm. Fix those (Part 8) before you swap algorithms; a good reward makes PPO look brilliant and a bad one makes every algorithm look broken.

EXERCISE 7.1

Train PPO on `LunarLander-v2` to a positive average return, then on a continuous-control env (`Pendulum-v1`) — note that the *same* PPO code handles both discrete and continuous actions. Then train SAC on `Pendulum-v1` and compare sample efficiency (reward vs timesteps). Draw your own conclusion about when each is worth it.

Part 8 · Reward Design — Where RL Projects Live or Die

If you remember one chapter, make it this one. The algorithm is rarely why an RL project fails; the **reward** is. The agent will do *exactly* what you reward — which is almost never quite what you meant. Designing that signal is the real engineering work of RL.

8.1 The agent optimises the reward, not your intent

This is the iron law: an RL agent is a relentless, literal-minded optimiser of the number you give it. If there's a loophole — a way to score reward without doing the task — it *will* find it. This is called **reward hacking**, and it's endless and creative.

DIAGRAM

You reward: "+1 per second the pole stays up"
Agent learns: exploit a simulator bug that freezes the pole. Reward: maxed. Task: failed.

ENGINEER'S MENTAL MODEL

Treat the reward like a metric your team is bonused on: whatever you measure, you get — including the perverse version. "Reward the number of tickets closed" and tickets get closed without being fixed. RL removes all the human judgement that normally stops that, so the loophole gets exploited to the limit. Design rewards like you're defending against an adversary who shares your codebase.

8.2 Sparse vs shaped rewards

- **Sparse reward** — reward only at the goal (+1 for winning, 0 otherwise). Honest and unhackable, but the agent may wander forever without ever stumbling onto the reward to learn from.
- **Shaped reward** — add intermediate hints (+0.1 for moving toward the goal). Learns faster, but every hint is a new loophole and can steer the agent to optimise the *hint* instead of the goal.

The craft is adding just enough shaping to make learning tractable without changing what "winning" means.

8.3 Practical rules

- **Reward outcomes, not behaviours** where you can — reward reaching the goal, not a specific way of moving, or the agent won't find better ways.
- **Keep magnitudes sane** — one huge term drowns the rest; normalise.

- **Penalise the costs you actually care about** (time, energy, risk) explicitly, or the agent will ignore them.
- **Watch what the agent actually does**, not just the reward number — a climbing reward with absurd behaviour means it found a loophole.

COMMON MISTAKE

Over-shaping. Developers pile on intermediate rewards to speed learning and accidentally define a *different* task — the agent maximises the proxy (hug the wall, jitter in place to farm a movement bonus) and never does the real thing. Start with the **sparsest reward that could work**, add shaping only where learning genuinely stalls, and re-check behaviour after each addition.

WHEN NOT TO TRUST A GREAT REWARD CURVE

A beautiful, rising reward curve is *not* proof of success — it's proof the agent maximised your number. If you haven't watched the actual behaviour (rendered episodes, real trajectories), you don't know whether it solved the task or hacked the reward. Always inspect behaviour, not just the plot.

EXERCISE 8.1

Take `MountainCar-v0` (famously hard with its sparse reward). First train PPO on the raw sparse reward and watch it struggle. Then add a small shaping term (e.g. reward for gaining height/velocity) and see learning speed up. Finally, deliberately over-shape it and find a behaviour that farms your bonus without reaching the flag — you'll have reward-hacked your own agent, which is the lesson.

Part 9 · Evaluation, Debugging & Stability

RL is the least stable corner of machine learning. The same code with a different random seed can succeed or flatline. Evaluating and debugging agents therefore takes more discipline than supervised learning, where a fixed test set gives you a clean number. This chapter is the survival kit.

9.1 Evaluate separately from training

During training the agent is exploring (acting randomly some of the time), so the training reward understates its real ability. **Evaluate on separate episodes with exploration off** (`deterministic=True`), averaged over many runs:

PYTHON

```
from stable_baselines3.common.evaluation import evaluate_policy
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=30)
print(mean_reward, "+/-", std_reward)
```

The **standard deviation matters as much as the mean** — a high mean with huge variance is an unreliable agent, not a good one.

9.2 Seeds and variance

Run every experiment with **several random seeds** (3-5 minimum) and report the spread. A single lucky seed proves nothing; RL results that don't report across-seed variance are close to meaningless.

ENGINEER'S MENTAL MODEL

In supervised learning a fixed test set makes runs comparable; in RL both the environment stochasticity and the exploration randomness inject noise, so one run is an anecdote. Treat a result as a **distribution over seeds**, not a point. Reporting "mean $\pm$ std over 5 seeds" is the RL equivalent of a proper test-set metric.

9.3 A debugging ladder

When an agent won't learn, work down this list — roughly in order of likelihood:

1. **The reward** — is it actually being given? Right sign, right magnitude, at the right time? (Part 8. This is the usual culprit.)
2. **The `done` /termination logic** — ending episodes wrongly poisons the value targets.

3. **The observation** — does the state actually contain what's needed to decide? (Markov property, Part 2.)
4. **Normalisation** — wildly-scaled observations or rewards destabilise the networks; normalise them.
5. **Hyperparameters / horizon** — learning rate, γ , and enough total timesteps (RL needs far more than you expect).
6. **Then, maybe, the algorithm.**

PRODUCTION TIP

Log to TensorBoard and always plot reward-vs-timesteps (SB3 does this with `tensorboard_log=`). The shape tells you a lot: flat = not learning (reward or observation bug); climbing then collapsing = instability (lower the learning rate, check the target network); noisy but rising = normal, train longer. The curve is your primary instrument.

9.4 Overfitting to the simulator

An agent can become excellent in its training environment and useless the moment anything differs — the **sim-to-real gap**. Guard against it with randomised environment parameters during training (**domain randomisation**) and by evaluating on variations the agent didn't train on.

COMMON MISTAKE

Declaring victory on the training environment. An agent that scores perfectly on the exact simulator it trained on may have memorised that simulator's quirks. Always evaluate on held-out variations (different seeds, perturbed dynamics), exactly as you'd never report training accuracy as a model's real performance in supervised learning.

EXERCISE 9.1

Train PPO on `LunarLander-v2` with 5 different seeds. Plot all five reward curves on one chart and compute mean $\pm$ std of `evaluate_policy` for each. Observe how much seeds differ. Then evaluate your best agent on a version with modified gravity/wind and measure how much performance drops — your sim-to-real gap in miniature.

Part 10 · RL in Production

Very few RL agents make it to production, and the ones that do usually run in a constrained, well-understood setting. This chapter is about that last mile: serving a trained policy, doing it safely, and the realistic patterns for using RL in a real system.

10.1 Serving a policy is easy — it's just inference

Once trained, a policy is a fast function `state → action`. Serving it is the same as serving any model: load it, expose an endpoint (FastAPI, from your Python book), and call `predict`.

PYTHON

```
from fastapi import FastAPI
from stable_baselines3 import PPO

app = FastAPI()
model = PPO.load("policy.zip")

@app.post("/act")
def act(state: list[float]):
    action, _ = model.predict(state, deterministic=True)
    return {"action": action.tolist()}
```

ENGINEER'S MENTAL MODEL

Training is the hard, expensive, offline part; **serving is trivial** — a forward pass, microseconds, no environment loop. The deployment story is identical to serving any ML model (your MLOps chapter applies unchanged). The architecture question is not *how* to serve, but *whether the world still matches training* — see drift below.

10.2 The safety problem: offline vs online

- **Offline (the safe default):** train in a simulator or on logged data, freeze the policy, deploy it read-only, and let it act. It never explores in production, so it never does something insane to "learn." This is how nearly all production RL works.
- **Online learning in production:** letting the agent keep exploring on live traffic is dangerous — exploration means deliberately trying suboptimal actions on real users/systems. Do this only with hard guardrails (action limits, a safe fallback policy, human oversight).

WHEN NOT TO LEARN ONLINE

Almost always. Live exploration on real customers, money, or hardware risks real harm for the sake of learning, and it's hard to bound. The safe pattern is **train offline, deploy frozen, retrain periodically on fresh logged data** — the same batch-retraining cadence as any production model. Online RL belongs to teams with simulators, guardrails, and a strong reason.

10.3 Drift — the world stops matching the simulator

A frozen policy assumes the environment stays like the one it trained on. When the real dynamics shift (user behaviour changes, the market moves, the hardware ages), the policy silently degrades — the RL version of model drift. Monitor realised reward in production and retrain when it sags.

10.4 Realistic production patterns

- **Sim-trained, deployed frozen** — robotics, control, games. Train in simulation, transfer carefully (domain randomisation, Part 9).
- **Offline RL** — learn a policy purely from a fixed log of past interactions, never interacting live. Increasingly practical when you have lots of logged behaviour and can't risk online exploration.
- **RL as a component** — the policy suggests, a rule-based system vetoes unsafe actions, a human approves high-stakes ones. RL rarely runs fully unsupervised.

PRODUCTION TIP

Ship a **non-RL baseline alongside the agent** and keep it as the fallback: a rule, a heuristic, or the previous system. Route to it whenever the agent's proposed action fails a safety check or confidence bar. This turns "the agent did something crazy" from an incident into a no-op, and gives you a clean A/B comparison of whether RL is actually beating the simple baseline (often it isn't — and that's worth knowing).

COMMON MISTAKE

Deploying an agent with no bound on its actions and no fallback, trusting that good training means safe behaviour. Out-of-distribution states *will* occur in production, and an unconstrained policy can respond to them with arbitrary actions. Always clamp actions to safe ranges and wire a fallback — treat the policy as a strong suggestion, not an unquestioned command.

EXERCISE 10.1

Take your best `LunarLander` PPO agent, save it, and serve it behind a FastAPI `/act` endpoint that takes a state and returns an action. Add a guardrail that rejects any action outside the valid range and falls back to a safe default. Then simulate drift by evaluating the frozen policy on a perturbed environment and decide, from realised reward, when you'd trigger a retrain.

Part 11 · Capstone & Cheat Sheet

11.1 Capstone — take one agent end to end

Pick an environment and go from nothing to a served, evaluated policy:

1. **Frame it** — confirm it's actually an RL problem (Part 1's five conditions). Define the MDP: state, actions, reward, γ (Part 2).
2. **Wrap it** — a stock Gymnasium env, or your own `Env` with `reset / step` (Part 3).
3. **Baseline** — train stock **PPO** and get a learning curve climbing (Part 7).
4. **Reward** — iterate on the reward; watch behaviour, not just the number, and catch any reward hacking (Part 8).
5. **Evaluate** — `evaluate_policy` over 30 episodes and **5 seeds**, report mean $\pm$ std, test on held-out variations (Part 9).
6. **Serve** — save the policy, expose `/act` via FastAPI, add an action guardrail and a non-RL fallback (Part 10).

A good target: solve `LunarLander-v2` (discrete) or `BipedalWalker-v3` (continuous) with PPO, evaluate honestly, and serve it. If you can do that, you can do RL.

11.2 Decision cheat sheet

DIAGRAM

```

Is it an RL problem?
  sequential? + reward? + cheap interaction? + no labelled actions? → maybe RL
  else → supervised / bandit / rule / optimiser (usually this)

Which algorithm?
  discrete actions, small state      → tabular Q-learning
  discrete actions, big state       → DQN (or PPO)
  continuous actions                → PPO, or SAC/TD3 for sample efficiency
  unsure                            → PPO (the robust default)
  "which of N options, one shot"    → contextual bandit, not full RL
  
```

11.3 Concept cheat sheet

DIAGRAM

state (s)	the situation the agent observes
action (a)	what it can do
reward (r)	scalar feedback after acting
return (G_t)	discounted sum of future rewards – the thing maximised
γ (gamma)	discount; 0=myopic, $\rightarrow 1$ =far-sighted (0.95–0.99 typical)
policy $\pi(a s)$	the strategy: what to do in each state (the goal)
$V(s)$ / $Q(s,a)$	value functions: expected return from a state / state-action
TD error	prediction gap you learn from: $r + \gamma \cdot \max Q(s') - Q(s,a)$
ϵ -greedy	explore with prob ϵ , else exploit

on/off-policy episode	SARSA learns what it did; Q-learning learns the best one run from reset to terminated/truncated
-----------------------	---

11.4 The five rules

1. **Most problems aren't RL problems.** Try a rule, a forecast, or a bandit first.
2. **Don't hand-roll algorithms.** Use Gymnasium + Stable-Baselines3; start with PPO.
3. **The reward is the program.** The agent optimises exactly what you reward — design it against loopholes, and watch behaviour, not just the curve.
4. **Report across seeds.** One run is an anecdote; RL results are distributions.
5. **Deploy frozen, with a fallback.** Train offline, clamp actions, keep a non-RL baseline to beat and to fall back to.

YOU'RE READY

You can now recognise an RL problem, frame it as an MDP, train value- and policy-based agents with the standard tools, design and debug rewards, evaluate honestly, and ship a policy safely — and, most importantly, tell when a forecast or a rule is the better tool. That judgement is what separates using RL from being impressed by it. Go teach something to act.